

SafeTransition: exact rational certification of transition safety for sustainability assessment

Amin Abaee^a

^a*Independent Researcher, ORCID: 0000-0002-0019-1842, amin_abaee@ut.ac.ir*

Abstract

SafeTransition, a Python library for certifying the transition safety of sustainability assessments, is presented. The library implements the assessment-operator framework of the companion manuscripts: typed floor constraints evaluated on exact reachability tubes, scalarized aggregate operators at arbitrary positive weights, and backward recursions over state and belief spaces. Infeasibility is returned as a Farkas certificate, a nonnegative multiplier vector with vanishing weighted constraint sum and negative weighted bound, re-verified exactly together with its infeasibility margin. All computations run in exact rational arithmetic with no runtime dependencies beyond the Python standard library; floating point appears only in rendered graphics. Twenty-four exact checks re-derive the resource-transition benchmark of the companion verification deposit, including the per-weight licensing thresholds $2/3$ and $3/2$ and the rescue threshold $1 - x$. A single-file dashboard renders the resulting readings. The library is distributed under the MIT licence.

Keywords: sustainability assessment, viability theory, transition safety, Farkas certificate, exact arithmetic, composite indicators, multi-criteria decision analysis

1. Introduction

Composite indicators occupy a central place in sustainability monitoring. Aggregates reduce heterogeneous floor constraints on income, biomass, and social outcomes to a single weighted reading, and that reading is what dashboards, statutory reviews, and public communications display. The practice rests on a long modelling tradition, and its governance follows established frameworks for good model development and evaluation practice [9]. It also rests on a mathematical step that is easy to audit and easy to overlook: a weighted sum is certified positive at every audited snapshot, and the system behind the snapshots is declared safe.

The companion manuscripts show that this step can fail, and characterize exactly when it fails. In the assessment-operator framework of Abaee [1], a noncompensatory typed operator — which requires every floor to hold on the full trajectory, not at endpoints alone — can reject every plan that a scalarized aggregate operator licenses at a given weight; the separation is exhibited on an explicit rational witness datum and propagated through backward recursions. A resource-transition benchmark realizes the witness as a regulated fishery transition and verifies every quoted number in exact rational arithmetic. Under incomplete observation, the obstruction calculus of Abaee [2] complements viability assessment from the other side: it certifies that no observation-based policy is viable, through finitely checkable certificates — the common-action obstruction with an explicit Farkas multiplier vector, the observation-fibre criterion, and a finite-horizon belief-space recursion

that is sound and complete in finite systems. These results are mathematical; they describe what should be computed. They do not yet describe a tool that computes it.

This is the gap SafeTransition addresses. Reading a composite dashboard today, an analyst can recompute the aggregate with ordinary floating-point tooling, but has no standard way to re-derive the safety verdict behind it, to check that an infeasible plan menu is infeasible by certificate rather than by solver exit code, or to see which weight ratios license which plans. Numerical viability and control toolboxes compute kernels and invariant sets to high precision; multi-criteria decision analysis packages aggregate and rank; formal verification systems certify finite-state models against specifications. What is missing in the intersection is a small, auditable layer in which the verdicts themselves — admissibility of a plan at a state, licensing at a weight, emptiness of a menu, emptiness of an epistemic kernel — are returned as exact rational objects that a third party can re-verify by hand or by machine.

This paper presents SafeTransition, a Python library that provides this layer. The library implements the five assessment operators and their accepted-state sets, the typed finite-graph backward recursion, the belief-space robust epistemic recursion, and the finitely checkable certificate family: exact rational Fourier–Motzkin elimination with provenance tracking, returning Farkas certificates and infeasibility margins; the finite common-action obstruction with a minimal conflicting subfamily; and the observation-fibre criterion. Every computation runs in exact rational arithmetic; floats appear only in rendered graphics. The library re-derives, in twenty-four exact checks, the resource-transition benchmark of the verification deposit, and renders the resulting readings as a single-file dashboard. It depends on nothing beyond the Python standard library, which makes the exactness claim structural rather than procedural: there is no numerical backend whose rounds of approximation would need to be audited.

The paper is a software description. Section 2 states the mathematical basis as implemented, with proofs delegated to the companion manuscripts. Section 3 describes the software architecture and the exact-arithmetic discipline. Section 4 illustrates the library on the benchmark, on the dashboard readings, and on the partial-observation demonstrations. Section 5 positions the library relative to existing software, and Section 6 records availability and requirements. Section 7 concludes.

2. Mathematical basis

2.1. Transition data and assessment operators

A typed transition datum is a phase state $z = (q, x, s_1, \dots, s_n)$ with a phase flag q , fund coordinates x , and typed floors s_i normalized to zero; a finite action set; a set of disturbances with, for each action and disturbance, an exact tube $\text{Tube}(a, d)$ — the set of states visited over the review period, given here as piecewise-linear coordinate paths on breakpoints at which every visited extreme is attained; a transition-safe set S ; and a destination set G reached under an endpoint reset. Actions in the library declare offsets on the breakpoints, together with an action-indexed worst-case disturbance, so that each tube is the exact visited set and no outer approximation enters.

Five operators are distinguished, following Abaee [1]. For a state z , the noncompensatory typed operator admits action a when every floor holds on $\text{Tube}(a, d)$ for every disturbance d and the successor lies in G ; the scalarized aggregate operator replaces the coordinate-wise condition by $w \cdot s \geq 0$ at weight w in the positive weight cone; the exact-tube physical and endpoint-only

physical operators weaken the constraint set; and the typed-endpoint operator evaluates floors at endpoints only. Because endpoints are contained in tubes and typed sets in physical sets, the chain

$$E_{\text{typ}}(z) \subseteq E_w(z) \subseteq E_{\text{tube,phys}}(z) \subseteq E_{\text{end}}(z)$$

holds for every positive weight, and the library verifies it exactly at any state on request (Section 3.3). The accepted-state set $\mathcal{V}[E] = \{z : E(z) \neq \emptyset\}$ and the compensatory accepted set $\mathcal{V}_{\text{weak}} = \bigcap_w \mathcal{V}_w$ are derived from the operators; the witnessed separation $\mathcal{V}_{\text{typ}} \subseteq \mathcal{V}_{\text{weak}}$, strict in general, is what a composite dashboard cannot see (Section 4.1).

2.2. Recursions

Two backward recursions are implemented. The typed recursion propagates viability backwards through an explicit state–action graph: a state is k -step viable when some action keeps every visited state inside the safe set and lands in the $(k - 1)$ -step viable set. On explicit graphs with constant-time predicates the recursion is polynomial in graph size and horizon. The belief-space recursion addresses partially observed systems, following the robust epistemic construction of Abaee [2]: states carry observation labels, an action is admissible for a belief when no state in the belief’s fibre enters a violation en route, and beliefs are updated to the labels reachable under the action. The recursion returns the sets $\mathcal{W}_k$ of k -step viable beliefs and is sound and complete in finite systems: a belief outside $\mathcal{W}_N$ admits no viable observation-based policy over the horizon. Post-observation recourse failures — beliefs in $\mathcal{W}_1 \setminus \mathcal{W}_2$ — are the signature output (Section 4.3).

2.3. Certificates

Infeasibility is certified, not merely reported. For a stacked rational system $Au \leq b$, the library runs Fourier–Motzkin elimination with provenance tracking: every derived row carries its multiplier vector over the original rows, and when a contradiction $0 \leq c$, $c < 0$ is derived, the normalized multipliers $\lambda \geq 0$ with $\lambda^\top A = 0$ and $\lambda^\top b < 0$ are returned as a Farkas certificate, together with the infeasibility margin $-\lambda^\top b > 0$. The certificate is re-verified exactly before being returned. The finite common-action obstruction fires when the safe-action sets of pairwise-compatible states intersect emptily while each is nonempty — every state is individually viable under full information, yet no observation-based policy is viable; the library reports a minimal conflicting subfamily. The observation-fibre criterion decides whether an exact observation-only certifier can exist at all: one exists if and only if safe-set membership is constant on observation fibres, and a violating fibre is reported otherwise.

3. Software description

3.1. Architecture

The library is organized as nine modules (Table 1). The `datum` module defines the transition-data interface and ships one fully specified instance, the witness datum of the companion manuscript, with destination gain $e = (1/4, 1/4)$ and rescue cost $c = 1$. Custom data subclass the interface. The `operators`, `recursion`, and `certificates` modules implement Section 2; the `indicators` and `dashboard` modules produce the readings of Section 4.2; the `benchmark` module carries the twenty-four exact checks of Section 4.1. A command-line interface exposes `verify`, `report`, and `demo`.

Module	Contents
rational	exact rational helpers (interpretation, rendering)
datum	typed transition data; the witness datum instance
operators	five operators, accepted-state sets, chain check
recursion	typed finite-graph recursion; belief-space recursion
certificates	Fourier–Motzkin with provenance; Farkas verification; common-action obstruction; fibre criterion
indicators	licensing thresholds, rescue threshold, blindness alarm
dashboard	single-file HTML rendering (inline SVG/CSS)
benchmark	twenty-four exact checks; verified schedule values
cli	verify / report / demo

Table 1: Module organization of SafeTransition 1.0.0 (1 269 source lines, 283 test lines).

3.2. The exact-arithmetic discipline

Every assessment, recursion, and certificate is computed in exact rational arithmetic. Rationals enter either as integers, as decimal strings, or as explicit numerator–denominator pairs; no implicit conversion from binary floating point exists anywhere in the library, so an input’s exactness is structural, not a matter of reviewer discipline. Floats occur in exactly two places, both rendering: SVG geometry in the dashboard and matplotlib coordinates in figure scripts. Checks never read floats. The distinction is the library’s design-centre claim: outputs are verifiable, not merely computed. A Farkas certificate, a licensing threshold, or a benchmark check can be re-verified by any rational-arithmetic implementation — including pencil and paper — and the test suite exercises exactly this: every certificate returned by the eliminator is re-verified against its defining identities before being returned, and the benchmark checks re-derive each deposited value symbolically (for example $\sigma(16/5) = 1088/125$ for the sustained-yield quota of the Schaefer realization).

3.3. Interfaces and complexity

The application programming interface mirrors the mathematics: **E(datum, z, mode, w)** returns the action set of an operator at a state; **check_chain** verifies the operator chain exactly; **belief_backward** returns the belief-viability sets $\mathcal{W}_k$; **certify_polyhedron** returns a feasibility verdict with its certificate; **compute** returns the dashboard readings at a state, weight, and read plan. The typed recursion on explicit graphs is polynomial in graph size and horizon; the Fourier–Motzkin eliminator is exponential in the worst case, as any exact general method is, which is why the certificate family targets the small stacked systems that transition menus produce (a handful of constraints; the witness instances eliminate in milliseconds). Belief enumeration bounds itself (4096 reachable beliefs) and raises rather than drifting.

4. Illustrative examples

4.1. The resource-transition benchmark

The benchmark realizes the witness datum as a regulated fishery transition: stock biomass with Schaefer surplus $\sigma(B) = rB(1 - B/K)$ ($r = 4$, $K = 10$, $B_{\text{lim}} = 2$), a pulse-plus-closed-season quota plan, a mid-season heatwave strike, and a staged rebuild financed by a fund buy-back. Running **safetransition verify** executes twenty-four exact checks that re-derive the deposited verification values: the witness datum and tube tables; quota admissibility on both disturbance

Artifact	Result
Benchmark exact checks (<code>safetransition verify</code>)	24/24 pass
Unit and integration test suite	27/27 pass
Runtime of checks + tests (laptop CPU)	under 1 s each
Farkas certificate, stacked menu window	$\lambda = (1/2, 1/2)$, margin $1/10$, re-verified
Belief recursion, recourse-failure system	root belief in $\mathcal{W}_1 \setminus \mathcal{W}_2$
Source / test lines	1 269 / 283

Table 2: Verification artifacts shipped with SafeTransition 1.0.0. Every check and certificate is evaluated in exact rational arithmetic.

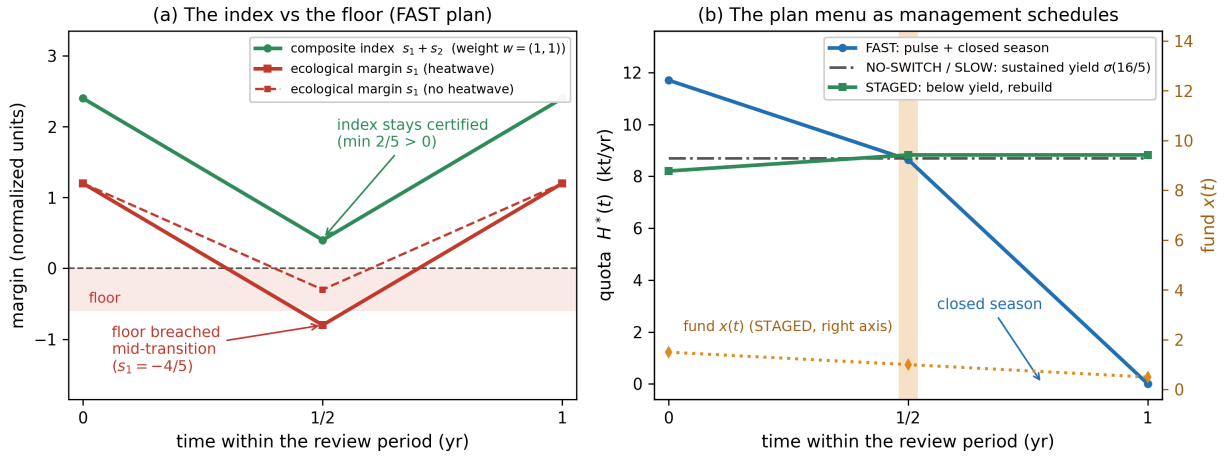


Figure 1: The resource-transition benchmark, regenerated from the library's exact checks. (a) The composite index at $w = (1, 1)$ stays certified (min $2/5 > 0$) along the licensed plan while the ecological margin s_1 is breached mid-transition ($-4/5$ adverse, $-3/10$ benign): the separation the typed operator exposes. (b) The plan menu as management schedules: FAST (pulse 1463/125 plus closed season), SLOW/NO-SWITCH (sustained yield 1088/125), and STAGED (below yield, rebuild to 69/20), with the fund schedule $x(t)$ under STAGED.

branches ($0 \leq H^* \leq 13$, peak 1463/125); exact tracking of the benign branch and conservatism of the certified tubes for the nonlinear realization ($\sigma \geq 528/125 \geq 4$ on the adverse recovery leg); the index blindness of the composite reading at $w = (1, 1)$, whose minimum stays at $2/5 > 0$ along the licensed plan while the ecological floor reaches $-4/5 < 0$ mid-transition; the licensing thresholds $\rho_1 = 2/3$ and $\rho_2 = 3/2$, cross-checked against the thresholds derived directly from the tube geometry; and the rescue threshold $\kappa^* = 1 - x$ on the non-typed-viable region, with shortfall $1/2$ at the false-positive witness and full financing at the rescue witness $x = 3/2$. All twenty-four checks pass in under a second on a laptop; Table 2 collects the verification artifacts.

Figure 1 shows the benchmark panels regenerated from the same verified values. Panel (a) is the separation a composite index cannot see: the weighted reading stays certified over the full review period while the floor it aggregates is driven below zero mid-transition. Panel (b) is the plan menu as management schedules: the pulse-plus-closed-season quota, the sustained-yield alternative at $\sigma(16/5) = 1088/125$, and the staged rebuild below yield while the fund finances the buy-back.

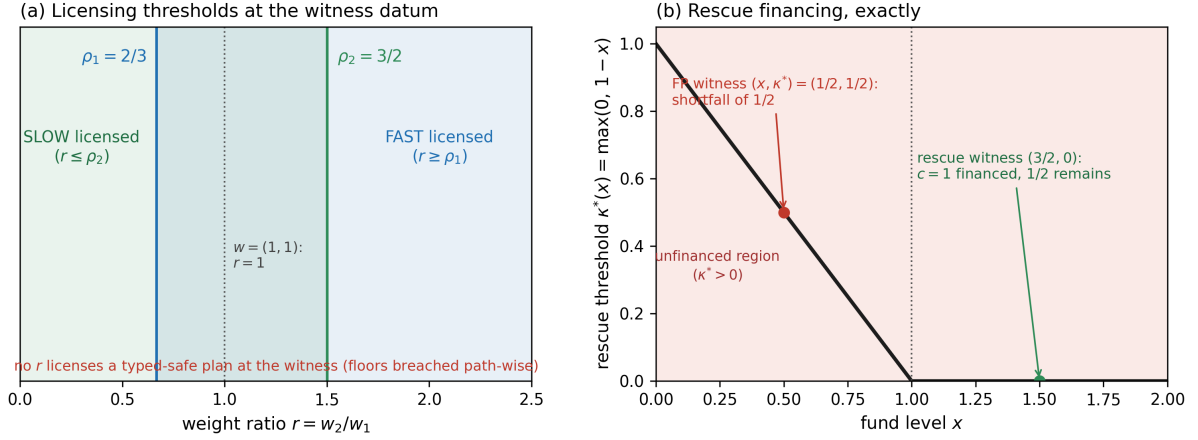


Figure 2: Dashboard readings at the witness datum, exact. (a) Per-weight licensing: the aggregate operator licenses FAST at weight ratios $r \geq \rho_1 = 2/3$ and SLOW at $r \leq \rho_2 = 3/2$; the standard equal-weight reading $r = 1$ sits inside the band where both are licensed and no ratio licenses a typed-safe plan. (b) Rescue financing: $\kappa^*(x) = \max(0, 1 - x)$; the false-positive witness $(1/2, 1/2)$ carries a shortfall of $1/2$, the rescue witness $(3/2, 0)$ finances the buy-back with $1/2$ remaining.

4.2. Dashboard readings

The `compute` interface returns, at a state and weight, the per-plan floor minima under each plan’s characteristic disturbance, the composite index along the read plan, the licensing verdicts at the weight ratio $r = w_2/w_1$, the rescue threshold, and the index-blindness alarm, which fires exactly when the composite minimum stays positive while a floor minimum is negative. At the witness state $(x, s_1, s_2) = (1/2, 6/5, 6/5)$ with $w = (1, 1)$, the alarm fires: the readings reproduce the benchmark separation as a dashboard statement (Fig. 2). The `dashboard` module renders the readings, the benchmark status, and any certificates as a single HTML file with inline SVG and CSS and no external resources, so the file can be attached to a review, emailed, or archived with a statutory record and remains readable offline.

4.3. Partial observation and certificates

The bundled demonstrations exercise the certificate family on the companion manuscript’s instances. The three-state recourse failure runs the belief-space recursion on a system whose root belief is one-step viable and exits at the second step under every observation-based policy: the root belief lies in $\mathcal{W}_1 \setminus \mathcal{W}_2$, witnessed only by the recursion. The observation-fibre criterion reports the merged fibre on which safe-set membership varies, ruling out any exact observation-only certifier. The stacked menu window $u \leq 2/5$, $-u \leq -3/5$ returns the Farkas certificate $\lambda = (1/2, 1/2)$ with infeasibility margin exactly $1/10$, re-verified against $\lambda^\top A = 0$ and $\lambda^\top b < 0$ before being returned; the certificate is the same object the obstruction calculus attaches to the empty intersection of compatible states’ safe menus. The pooled-kernel identity of the deposited double-integrator instance, weights $(3/8, 5/16, 5/16)$ with $\sum_j \lambda_j n_j = 0$, is checked exactly.

5. Relation to existing software

SafeTransition complements three established families of tools, and borrows from each. Numerical viability kernels and reachability tools (for the viability-theoretic tradition, and for general-

purpose simulation frameworks in the environmental modelling literature, e.g. Raven [5] and FloPy [3]) compute set-valued safety objects to high floating-point precision with impressive scalability; they answer *how large is the safe set*, where SafeTransition answers *is this verdict verifiable*, for the small, structured, rational instances that assessment menus produce. Multi-criteria and sensitivity analysis packages — the SAFE toolbox [10], SALib [7], SPOTPY [8] — share the library’s workflow-oriented design: model-independent interfaces, worked examples, and benchmark functions; SPOTPY’s demonstration that one package can carry an entire family of methods with a minimal amount of glue code is the presentation standard SafeTransition aims at. Modular environmental simulators with public source code and regression suites — Pywr [11] and airGR [4] — inform the availability discipline: permissive licensing, distribution through package indexes, and a test suite executed on continuous integration. None of these families returns certificates; none computes in exact arithmetic. The comparison is functional, not competitive: the families operate at scales and purposes for which floating-point computation is the right engineering choice, and the exact layer becomes decisive precisely where verdicts enter statutory or adversarial settings and must be re-derivable by third parties.

6. Availability and requirements

Name: SafeTransition

Owner and contact: Amin Abaee; amin_abaee@ut.ac.ir

Year first available: 2026

Hardware required: none beyond a standard computer; no specialized hardware

Software required: Python ≥ 3.9 ; no packages beyond the standard library (matplotlib and Pillow required only for regenerating figures)

Program language: Python

Program size: 1 269 source lines, 283 test lines

Availability and cost: open source under the MIT licence; source, tests, and examples are archived in the verification deposit [6] and mirrored in the project repository; no cost

The test suite (27 tests) covers the benchmark, the operator chain and witnessed separation on the witness datum, the certificate family on the deposited instances, the belief-space recursion on the recourse-failure system, and the dashboard’s self-containment; it runs in under a second. The dashboard example regenerates the single-file HTML from the library.

7. Conclusions

SafeTransition turns the assessment-operator framework and the obstruction calculus of the companion manuscripts into auditable software: typed operators evaluated on exact tubes, recursions over state and belief space, and infeasibility returned as re-verifiable Farkas certificates with exact margins. The library’s scope is deliberate: it certifies verdicts on small, explicit, rational instances, and it makes no predictive claim — the theorem-level guarantees are those of the

companion manuscripts, and on continuous or belief-continuous systems the graph construction itself is the modelling step. Within that scope, the tool closes a practical gap: the readings that composite dashboards display, and the nonviability verdicts that justify tightening a plan menu, can now carry their certificates with them. Extensions under way are continuous-state data with certified enclosures and additional certificate families from the obstruction calculus; the interface intentionally leaves room for both.

CRediT authorship contribution statement

Amin Abaee: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Declaration of generative AI in the writing process

During the preparation of this work the author used an AI assistant in order to assist with drafting and code development. After using this tool/service, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

Acknowledgements

Not applicable.

References

- [1] Abaee, A., 2026a. Aggregate Indices and Transition Safety: A Quantifier-Order Separation Between Scalarized and Coordinate-Wise Feasibility. Manuscript submitted for publication. Verified companion deposit: <https://doi.org/10.6084/m9.figshare.33764023>.
- [2] Abaee, A., 2026b. An Obstruction Calculus for Viability under Incomplete Observation. Manuscript submitted for publication.
- [3] Bakker, M., Post, V., Langevin, C.D., Hughes, J.D., White, J.T., Starn, J.J., Fienen, M.N., 2016. Scripting MODFLOW model development using open source Python and FloPy. *Groundwater* 54 (5), 733–739. <https://doi.org/10.1111/gwat.12413>.
- [4] Coron, L., Thirel, G., Delaigue, O., Perrin, C., Andréassian, V., 2017. The suite of lumped GR hydrological models in an R package. *Environmental Modelling & Software* 94, 166–171. <https://doi.org/10.1016/j.envsoft.2017.05.002>.
- [5] Craig, J.R., Brown, G., Chlumsky, R., Jenkinson, R.W., Jost, G., Lee, K., Mai, J., Serrer, M., Sgro, N., Shafii, M., Snowdon, A.P., Tolson, B.A., 2020. Flexible watershed simulation with the Raven hydrological modelling framework. *Environmental Modelling & Software* 129, 104728. <https://doi.org/10.1016/j.envsoft.2020.104728>.
- [6] Abaee, A., 2026. Verification code and figure pipeline for Aggregate Indices and Transition Safety [data set]. figshare. <https://doi.org/10.6084/m9.figshare.33764023>.
- [7] Herman, J., Usher, W., 2017. SALib: An open-source Python library for sensitivity analysis. *Journal of Open Source Software* 2 (9), 97. <https://doi.org/10.21105/joss.00097>.
- [8] Houska, T., Kraft, P., Chamorro-Chavez, A., Breuer, L., 2015. SPOTting model parameters using a ready-made Python package. *PLOS ONE* 10 (12), e0145180. <https://doi.org/10.1371/journal.pone.0145180>.
- [9] Jakeman, A.J., Letcher, R.A., Norton, J.P., 2006. Ten iterative steps in development and evaluation of environmental models. *Environmental Modelling & Software* 21 (5), 602–614. <https://doi.org/10.1016/j.envsoft.2006.01.004>.
- [10] Pianosi, F., Sarrazin, F., Wagener, T., 2015. A Matlab toolbox for global sensitivity analysis. *Environmental Modelling & Software* 70, 80–85. <https://doi.org/10.1016/j.envsoft.2015.04.009>.
- [11] Tomlinson, J.E., Arnott, J.H., Harou, J.J., 2020. A water resource simulator in Python. *Environmental Modelling & Software* 126, 104635. <https://doi.org/10.1016/j.envsoft.2020.104635>.